PROJECT REPORT

API Security Framework: Configured Automated OWASP ZAP and Postman Collections to Check for BOLA Flaws

Project domain: API Security Testing
Focus: Broken Object Level Authorization (BOLA)
Institution branding: Sandip University logo supplied for this report

Abstract

This project proposes a repeatable API authorization-testing workflow focused on Broken Object Level Authorization (BOLA), also known as an object-level authorization failure. It combines a Python test-plan runner with complementary OWASP ZAP and Postman workflows. The Python component sends non-destructive GET/HEAD requests for two dedicated test identities and compares response status codes, then writes a JSON report. A successful response for both identities is treated only as a review signal; the API's authorization model and response semantics must be validated manually.

1. Introduction

APIs commonly expose resources identified by object IDs, account IDs, document IDs, or similar references. If a server authenticates a caller but fails to check whether that caller is authorized for the requested object, another user may be able to access data they should not see. This class of issue is commonly described as BOLA. Authorization must be enforced server-side for each object and operation.

2. Problem Statement

Manual API testing can overlook object-level authorization differences between users. A request that succeeds for the resource owner may also succeed for another identity because the API lacks an ownership check—or because the object is intentionally shared. A useful framework must compare controlled test cases, retain evidence, and avoid declaring a vulnerability from status codes alone.

3. Objectives

- Create a repeatable two-identity authorization test workflow.
- Use synthetic objects in an authorized staging API.
- Provide a Python utility to execute a small JSON test plan.
- Use OWASP ZAP and Postman as complementary testing tools.
- Produce structured reports and document manual validation steps.

4. Scope and Safety

The included Python tool supports only GET and HEAD methods and does not enumerate endpoints or mutate application data. It requires two dedicated bearer tokens supplied through environment variables. It rejects absolute URLs in individual test paths so credentials remain scoped to the configured base host. Testing must be authorized, preferably performed in staging, and must use synthetic records. Do not test against real users' private data.

5. Proposed Architecture

Component	Role
Postman collection	Organizes positive owner-access and negative cross-user authorization cases with separate test environments.

Component	Role
OWASP ZAP	Supports authorized API exploration and security testing within the explicitly defined scope.
Python test runner	Executes configured GET/HEAD requests with two identities and compares status codes.
JSON report	Records test IDs, statuses, response fingerprints, review signals, and limitations.

6. Methodology

1) Confirm written authorization and define the API scope. 2) Create two dedicated test users and synthetic objects. 3) Define owner and alternate-user requests in a JSON plan and Postman collection. 4) Configure ZAP to the approved staging API scope. 5) Run the Python test plan using environment-based tokens. 6) Compare owner and alternate-user results. 7) Verify the API's intended sharing model and inspect sanitized server logs. 8) Document verified issues, remediate server-side authorization checks, and retest.

7. BOLA Test Logic

For a configured object belonging to identity A, the runner sends the same request as identity A and identity B. If A receives a 2xx response and B does not, the result is recorded as an expected access-control signal. If both receive 2xx responses, the result is marked REVIEW_POSSIBLE_BOLA. This does not prove a flaw: public objects, shared records, tenant-level permissions, and other valid policies may explain the result. If the owner request fails, the test is inconclusive.

8. Example Execution

```
python api_security_framework.py --base-url https://staging-api.example.test --plan test_plan.example.jso
```

Set API_TEST_TOKEN_A and API_TEST_TOKEN_B in the shell environment before running. Never place real tokens in the repository or report.

9. Evidence and Reporting

Record the test case ID, object ownership, identities used, expected authorization behavior, response status codes, sanitized evidence, and remediation status. The included report records SHA-256 fingerprints of response bodies rather than including body contents. If evidence requires response excerpts, capture only synthetic data and redact secrets.

10. Expected Outcomes

- A documented process for positive and negative object-authorization tests.
- Repeatable comparisons between resource owners and alternate identities.
- JSON output suitable for review and regression tracking.
- Better collaboration between developers and security testers using ZAP and Postman.

11. Limitations

This is a small educational template, not a complete API scanner. It does not import or execute Postman collections, invoke ZAP APIs, discover endpoints, test every HTTP method, or prove exposure of sensitive data. Token formats and API response behavior vary. Status codes and body fingerprints are indicators only; manual validation and server-side review are required.

12. Conclusion

The API Security Framework demonstrates a controlled way to assess object-level authorization by comparing the same API request under two test identities. The Python runner, JSON test plan, and ZAP/Postman workflow provide a starting point for repeatable testing. The most important outcome is a validated authorization decision—not merely a successful HTTP response.

References and Further Reading

- OWASP API Security Top 10 — Broken Object Level Authorization (API1).
- OWASP ZAP documentation — API testing and automation.

- Postman documentation — collections, environments, and test scripts.
- OWASP Web Security Testing Guide — authorization testing.

Prepared as an educational project template. Replace placeholder paths with synthetic staging objects and record actual authorized test results before submitting as a completed implementation.